

Track 13 — Wer die Rolle vergibt, und wer sie wieder wegnehmen kann

Datum: 2026-08-28 **Ausgangsstand:** 6efec2a (Track 12), 1168 Tests **Endstand:** 1189 Tests (21 neu), Typecheck grün

Vorgehen

Die im Auftrag benannten Bereiche wurden der Reihe nach gegen die tatsächlichen Quellen gelesen. Der überwiegende Teil war nach den Tracks 8–12 in Ordnung; die Prüfung ist trotzdem hier festgehalten, damit der nächste Track nicht dieselbe Strecke noch einmal läuft.

Ohne Befund:

- **Stripe-Webhook-Signatur.** `stripe.webhooks.constructEvent` über den Rohtext; fehlender Header und fehlendes Secret werden getrennt abgewiesen (`src/app/api/stripe/webhook/route.ts:633`).
- **Betrags-Manipulation im Checkout.** Kein Zweig nimmt einen Betrag aus dem Request. `booking` liest `price_cents` aus der eigenen Buchung, `rental total_cents` aus der eigenen Miet-Buchung, `product_order` die `unit_price_cents` der Positionen, `subscription` die Price-ID aus der Konfiguration. Alle vier Zweige sind zusätzlich auf den Aufrufer eingegrenzt.
- **Preisbildung Miete.** `POST /api/rental-bookings` nimmt nur `equipmentId` und Zeitraum; der Betrag entsteht serverseitig aus `rental_equipment`.
- **Negative Preise.** `rental-equipment` (POST und PATCH) validiert über `z.coerce.number().int().min(0).max(10_000_000)`, und ein Objekt mit Tagespreis 0 kann nicht online gehen.
- **Salon-Fremdzugriff.** `requireOwnedSalon` / `requireOwnedEquipment` (`src/modules/rentals/listing.service.ts`) lösen über `salons.owner_id` auf; `PATCH /api/provider/salon` sucht den Salon über die eigene `owner_id` statt über eine ID aus dem Request.
- **Buchungs-Race.** `checkConflict` vor dem Insert, danach eine Nachprüfung mit totaler Ordnung (`losesSlotRace`), dazu der Zweig für `23P01`. Der Kunde kann seinen Preis nicht setzen — `price_cents` kommt aus `services`, und Leistung und Salon müssen zusammengehören.
- **Datei-Uploads.** MIME-Allowlist ohne SVG, 5-MB-Deckel, Pfad aus `crypto.randomUUID()`, Besitzprüfung vor dem Upload, privater Bucket mit Signed-URL-Redirect.
- **Cron-Endpunkte.** Alle drei über `isAuthorizedCron` mit zeitkonstantem Vergleich; fehlendes `CRON_SECRET` sperrt statt zu öffnen.
- **Rate-Limiting.** Middleware (60/min pro IP, 10/min auf `/api/auth`, 30/min auf `/api/availability`) plus eigene Limits auf `promote-admin`, `register-provider` und `/api/email`.

Der Befund liegt woanders — eine Ebene unter allem, was diese Routen prüfen.

Befunde

1 — P0: Die Rolle kam aus einem Feld, das dem Konto selbst gehört

`authorizeCredentials()` lud nach erfolgreicher Supabase-Anmeldung das Profil und hatte für zwei Fälle einen Rückfall — Lookup fehlgeschlagen und Profil nicht gefunden. Beide stellten die Session so aus:

```
role: (data.user.user_metadata?.role as string) || 'kunde',
```

`user_metadata` ist in Postgres `auth.users.raw_user_meta_data`. Dieses Feld gehört dem Konto und ist **mit dem öffentlichen Anon-Key** schreibbar:

```
supabase.auth.updateUser({ data: { role: 'super_admin' } })
```

`signup({ options: { data: ... } })` nimmt es schon bei der Registrierung entgegen — `/api/register-provider` schreibt dort selbst `role: 'anbieter'` hinein. `NEXT_PUBLIC_SUPABASE_URL` und `NEXT_PUBLIC_SUPABASE_ANON_KEY` stehen im ausgelieferten Bundle; der Aufruf braucht die Anwendung gar nicht.

Der Rückfall setzte genau eine Sache voraus: **einen Auth-Nutzer ohne Zeile in `profiles`**. Und genau den hat die Anwendung selbst erzeugt — siehe Befund 3. Der DB-Trigger `handle_new_user` schreibt für neue Konten fest `'kunde'` (Migration `20260316_fix_register_trigger`), die Rolle war also über den regulären Weg nicht wählbar. Über den verwaisten Nutzer schon.

Was daraus folgte: Jede Route unter `/api/admin/**` prüft ausschließlich `session.user.role` — Salon-Freischaltung, Rollenvergabe, Buchungsstatus, Refunds, MIS-Export mit Kundendaten, Newsletter-Versand.

Der Testfall dazu hat den Zustand nicht übersehen, sondern **festgeschrieben**:

```
it('fällt auf die Auth-Metadaten zurück, wenn noch kein Profil existiert', ...)
  expect(user).toMatchObject({ ... role: 'anbieter' })
```

— dasselbe Muster wie bei `ip_hash` in Track 12: grün, weil der Test die Schreibweise prüfte statt die Eigenschaft.

Jetzt: `maybeSingle()` statt `single()`, damit „kein Profil“ von „Abfrage fehlgeschlagen“ unterscheidbar ist. Kein Profil → die Zeile wird mit `role: 'kunde'` nachgezogen (das hat der Kommentar an der Stelle ohnehin behauptet, ohne dass je ein Profil entstanden wäre), und die Session bekommt `'kunde'`. Lässt sich das Profil nicht anlegen → kein Login. Aus `user_metadata` kommt nur noch der Anzeigename.

2 — P0: Rechteentzug war 365 Tage lang wirkungslos

Der `jwt`-Callback setzt `token.role` nur `if (user)` — also beim Login. Danach wird die Rolle nie wieder angefasst. Dazu:

```
session: { maxAge: 365 * 24 * 60 * 60, updateAge: 24 * 60 * 60 }
```

Der Rolling-Refresh stellt den Token alle 24 Stunden neu aus und übernimmt `token.role` dabei unverändert. Die Rolle beim Anmelden galt damit ein Jahr.

Praktische Folgen, alle drei real:

- **PATCH /api/admin mit action: 'user-role'** schreibt `profiles.role`. Der herabgestufte Admin blieb in seiner offenen Sitzung Admin. Es gab keinen Weg, jemandem Admin-Rechte zu entziehen, außer zu hoffen, dass er sich abmeldet.
- **is_active = false** (Sperrung, DSGVO-Löschung über `/api/account/delete`) wird in `authorizeCredentials` geprüft — das sperrt den *nächsten* Login. Ein bereits ausgestelltes Cookie kam daran vorbei.
- **Hart gelöscht Profil** (`/api/cron/hard-delete`) hinterließ eine Session, die weiter als ihr früherer Inhaber galt.

Jetzt entscheidet über Rolle und Zugang bei jedem Aufruf die Datenbank. `getSession()` (`src/modules/auth/session.ts`) hält den Token gegen `profiles` und überschreibt `session.user.role` mit dem gelesenen Wert; kein Profil, `is_active = false` oder `deleted_at` gesetzt heißt: keine Session. Der Token liefert nur noch die Identität.

Fail closed: Lässt sich der Kontostand nicht lesen, gibt es keine Session. Eine Rolle aus einer Quelle auszustellen, die gerade nicht antwortet, ist genau der Fehler aus Befund 1.

Ein Kontostand wird 15 Sekunden wiederverwendet, damit der Zugriff nicht an jeder Server-Component hängt; die Stellen, die Rolle oder Sperre selbst ändern (`/api/admin user-role`, `/api/setup/promote-admin`, `/api/account/delete`), werfen den Eintrag über `invalidateAccountState()` sofort weg.

Damit die Prüfung nicht zu umgehen ist, holen jetzt **alle** 21 Routen, die bisher direkt `auth()` aufrufen, ihre Session über `getSession()`. Ein statischer Test hält beides fest: kein Produktivcode liest eine Rolle aus `user_metadata`, und keine Route unter `src/app/api` importiert `auth` aus `auth.config` (Ausnahme: der NextAuth-Handler selbst).

Die Middleware prüft weiter gegen den Token. Sie ist die grobe Vorsortierung, nicht die Grenze — das steht so schon im Kommentar zu `adminPaths` seit Track 11 und bleibt richtig.

3 — P1: /api/register-provider erzeugte anmeldbare Konten ohne Profil

Schlug der Salon-Insert fehl, löschte der Handler das Profil und ließ den Auth-Nutzer stehen:

```
await admin.from('profiles').delete().eq('id', userId)
```

Das ist genau die Vorbedingung aus Befund 1 — und in den Metadaten dieses Nutzers steht durch den `signUp` darüber `role: 'anbieter'`, vom Kontoinhaber mit dem Anon-Key auf alles andere umschreibbar.

Der zweite Effekt: Das Aufräumen sollte laut Kommentar einen erneuten Versuch ermöglichen. Es tat das Gegenteil — das Auth-Konto blieb, die Adresse war für jeden weiteren Versuch mit „User already registered“ verbrannt, und der Betroffene hatte weder Salon noch Profil.

Jetzt wird zuerst das Auth-Konto gelöscht (`auth.admin.deleteUser`), und das Profil nur dann, wenn das gelungen ist — sonst entstünde der verwaiste Nutzer gerade durch das Aufräumen. Die

Registrierung ist damit tatsächlich wiederholbar.

4 — P2: Ein Datenbank-Aussetzer beim Login übersprang die Kontosperrung

Der Rückfall bei `profileError` stellte eine Session mit Rolle aus, obwohl der Kontostand ungelesen blieb — `is_active = false` wurde in diesem Pfad nie geprüft. Ein Timeout auf `profiles` genügte, um ein gesperrtes oder DSGVO-gelöshtes Konto wieder anmeldbar zu machen. Jetzt: kein Login.

5 — Nebenbefund: eine rote Testdatei, die nichts mit Code zu tun hatte

`src/__tests__/keine-erfundene-reputation.test.ts` war schon vor diesem Track rot (gegen `6efec2a` reproduziert): der dynamische Import in `beforeAll` zieht die halbe Modulkette mit und braucht kalt mehr als die 10 Sekunden Standard-Timeout. Elf Tests wurden dadurch übersprungen, ohne dass jemand etwas davon hatte. Der Hook hat jetzt einen eigenen Timeout.

Tests

21 neue Tests. Die Angriffe werden ausgeführt, nicht beschrieben.

`src/__tests__/e2e/rollen-eskalation.test.ts` (19)

- *Angriff 1 — selbst gesetzte Rolle.* Auth-Nutzer ohne Profil mit `user_metadata.role = 'super_admin'` bekommt `'kunde'`; das fehlende Profil wird mit `'kunde'` nachgezogen; scheitert das, gibt es keinen Login; ein Lesefehler auf `profiles` stellt keine Session aus; ein vorhandenes Profil schlägt widersprechende Metadaten.
- *Angriff 2 — veraltetes Cookie.* Token sagt `super_admin`, `profiles` sagt `kunde` → die Session trägt `kunde`, und `PATCH /api/admin` antwortet mit genau diesem Cookie 403 statt die Rolle zu ändern. Dazu: gesperrtes Konto, `deleted_at`, hart gelöscht Profil, Lesefehler — alle vier beenden die Sitzung; ein echter Admin bleibt Admin; eine Rolle fehlt im Token und wird aus der Datenbank geholt.
- *Kontostand-Cache.* Ein Lesezugriff im Fenster, sofortiger Neuzugriff nach `invalidateAccountState`, und das Herabstufen über die Admin-Route wirkt ohne Wartezeit.
- *Statische Absicherung.* Kein `user_metadata...role` im Produktivcode (Kommentare ausgenommen — beschrieben werden darf der Befund), keine API-Route mit direktem `auth`-Import.

`src/__tests__/e2e/provider-onboarding.test.ts` (2 neu) Der Fehlerzweig löscht auch das Auth-Konto; schlägt das fehl, bleibt das Profil stehen, damit kein verwaister Nutzer entsteht.

`src/__tests__/e2e/auth-flow.test.ts` (1 umgeschrieben) Der Test, der die Metadaten-Rolle festgeschrieben hatte, prüft jetzt die Eigenschaft: fehlendes Profil → `'kunde'`, und die Zeile wird angelegt.

Harness: `auth.admin.deleteUser` ergänzt (`_harness/fake-supabase.ts`), inklusive Schalter für den Fehlerfall. Eine Fremdschlüssel-Kaskade auf `profiles` behauptet der Nachbau bewusst nicht — sie ist von hier aus nicht prüfbar, und der Produktivcode räumt das Profil selbst ab.

Offen

- **Unverifizierte Salons sind sofort im Markt.** `/api/register-provider` setzt `is_active: false`, `is_verified: false`. Beide Felder werden auf der Mietstrecke **nirgends ausgewertet**: `GET /api/rental-listings` filtert nur `rental_equipment.is_available`, und `POST /api/rental-bookings` prüft den Salon-Status nicht. Wer sich registriert, kann sofort ein Inserat anlegen, erscheint in der öffentlichen Suche und nimmt echtes Geld entgegen. Ob das ein Freischalt-Tor bekommen soll, ist eine Produktentscheidung — ein Filter auf `is_active` würde jeden Salon aus dem Markt nehmen, dessen Flag heute nicht gesetzt ist, und wie viele das sind, ist von hier aus nicht lesbar.
- **`passwordMustChange` ist eine tote Sperre.** Die Middleware wertet `session.user.passwordMustChange` aus (`src/middleware.ts:383`) und leitet auf `/auth/change-password` um. Ge setzt wird das Feld nirgends — weder im `jwt` -Callback noch sonst im Repository. Der Zwang kann also nie auslösen.
- **Middleware-RBAC bleibt token-basiert.** Bewusst: die Nachprüfung braucht einen Datenbankzugriff, und die Middleware läuft vor jeder Seitenauslieferung im Edge-Runtime. Die Grenze ist die Route.
- **15 Sekunden Nachlauf.** Rollen- und Sperränderungen, die nicht über die drei bekannten Stellen laufen (etwa direkt im Supabase-Dashboard), wirken bis zu 15 Sekunden später.
- **`PATCH /api/provider/salon` validiert keine Werte.** Die Feld-Allowlist steht, aber Länge und Typ der Werte werden nicht geprüft — `opening_hours` nimmt beliebiges JSON, `name` beliebige Länge. Kein Rechteproblem, aber die einzige Schreibroute auf `salons` ohne Zod-Schema.